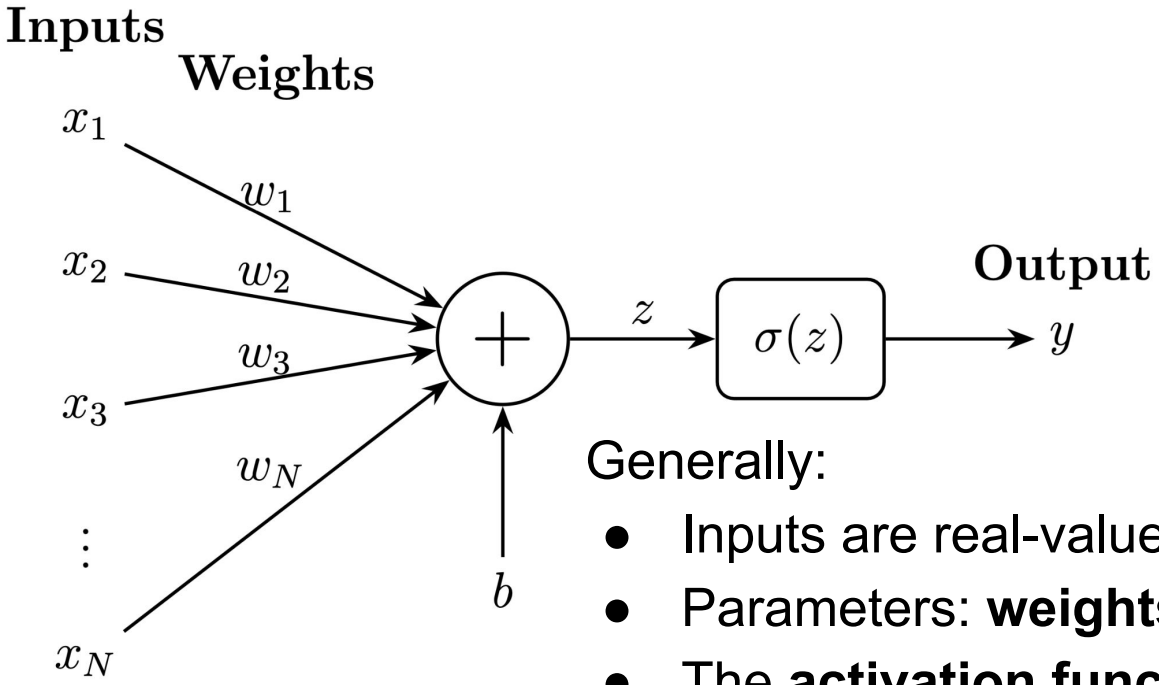
CS228: Introduction to Deep Learning

Lecture 4: learning

Instructor: Zhouxing Shi

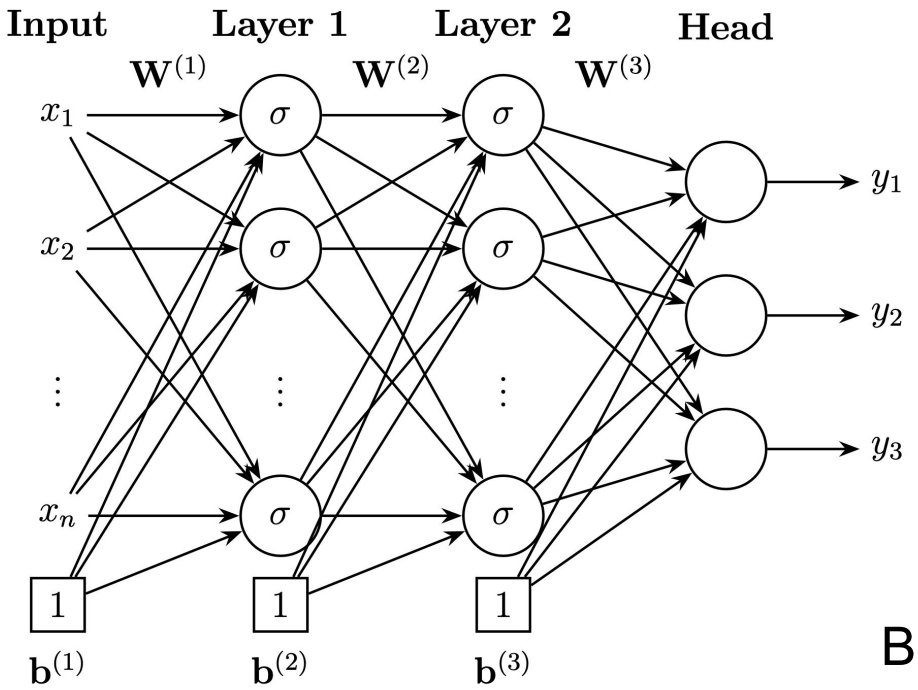
Perceptron unit



Generally:

- Inputs are real-valued
- Parameters: **weights** *w* and **bias** *b*
- The **activation function** is a nonlinear function (e.g., ReLU, Sigmoid, ...) and not necessarily a threshold function

MLP



$$\mathbf{y} = \mathbf{W}^{(3)}\sigma\left(\mathbf{W}^{(2)}\,\sigma\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)}\right) + \mathbf{b}^{(3)}$$

MLPs can model:

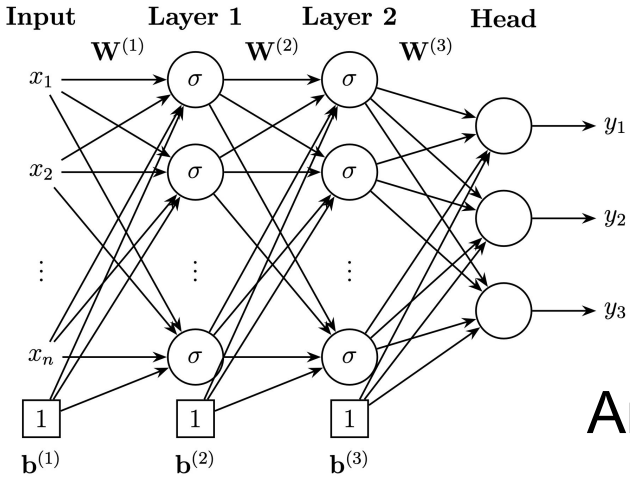
- Any boolean function
- Any classification boundary
- Any continuous valued function

(With a sufficient size)

Biases can be viewed as the weight of another input which is always 1
(we may omit biases sometimes)

Learning problem

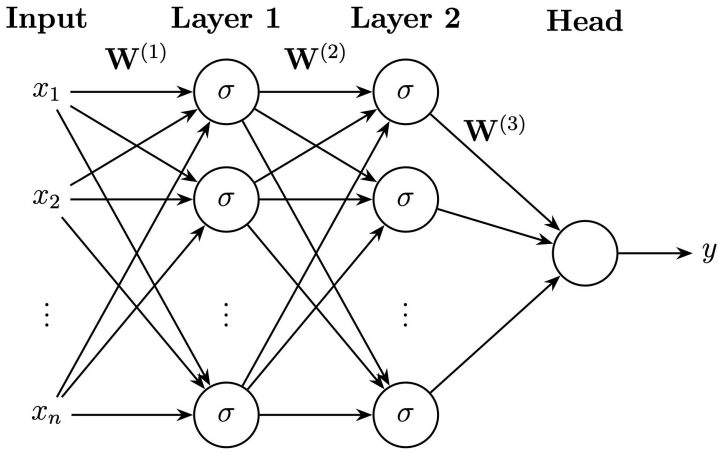
- Given the architecture of the network
- **Determine the values of the parameters**
- Such that the network **models a desired function**



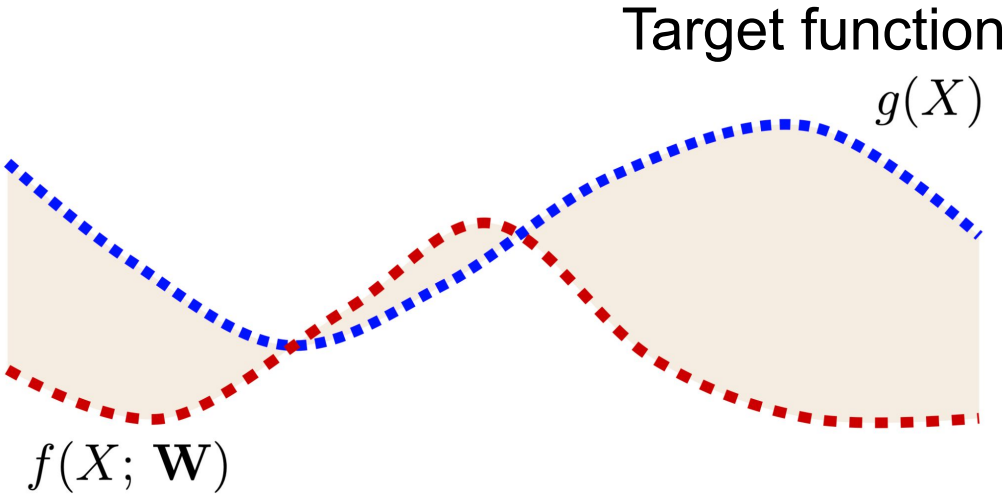
$$\mathbf{y} = \mathbf{W}^{(3)}\sigma\big(\mathbf{W}^{(2)}\,\sigma\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big) + \mathbf{b}^{(2)}\big) + \mathbf{b}^{(3)}$$

Architecture: feed-forward, fully-connected

Learning problem

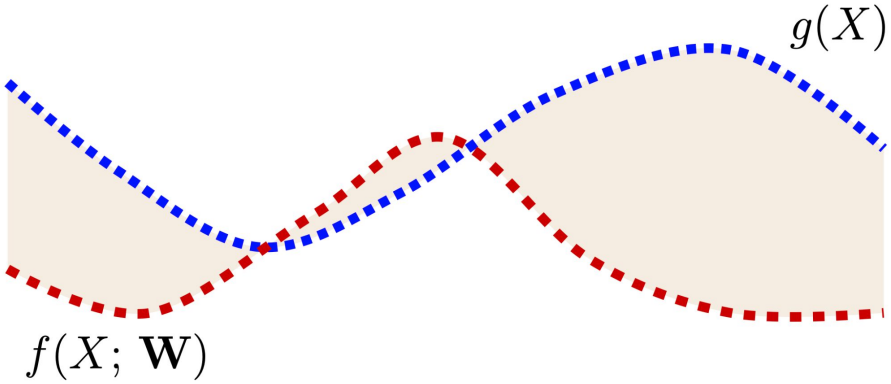


$f(X;\ \mathbf{W})$



Goal: estimate parameters to minimize the approximation error

Learning problem

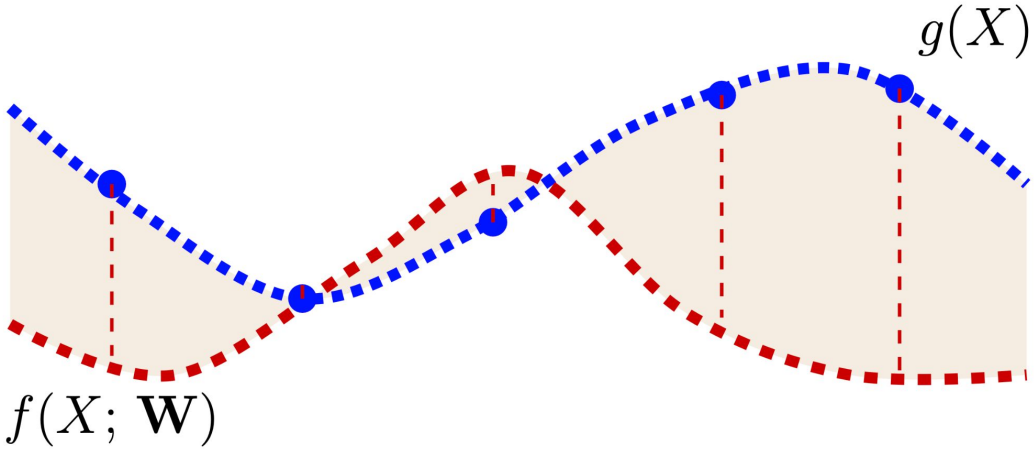


$$\text{total_error}(\mathbf{W}) = \int_{-\infty}^{\infty} \text{error}(f(X;\,\mathbf{W}),\, g(X))\, dX$$

$$\hat{\mathbf{W}} = \arg\min_{\mathbf{W}} \;\; \text{total_error}(\mathbf{W})$$

In reality $g(X)$ is not fully known

Learning from samples



Empirical error: average error over training examples $(X_i,\ g(X_i))$

$$\text{empirical_error}(\mathbf{W}) = \sum_{i=1}^{n} \text{error}(f(X_i;\ \mathbf{W}), g(X_i))$$

$$\hat{\mathbf{W}} = \arg\min_{\mathbf{W}} \ \text{empirical_error}(\mathbf{W})$$

Generalization: hopefully the error is also small on other inputs

Classification task

- Binary classification: $f(X_i;\ \mathbf{W}), y_i \in \{0, 1\}$

  or -1 and +1

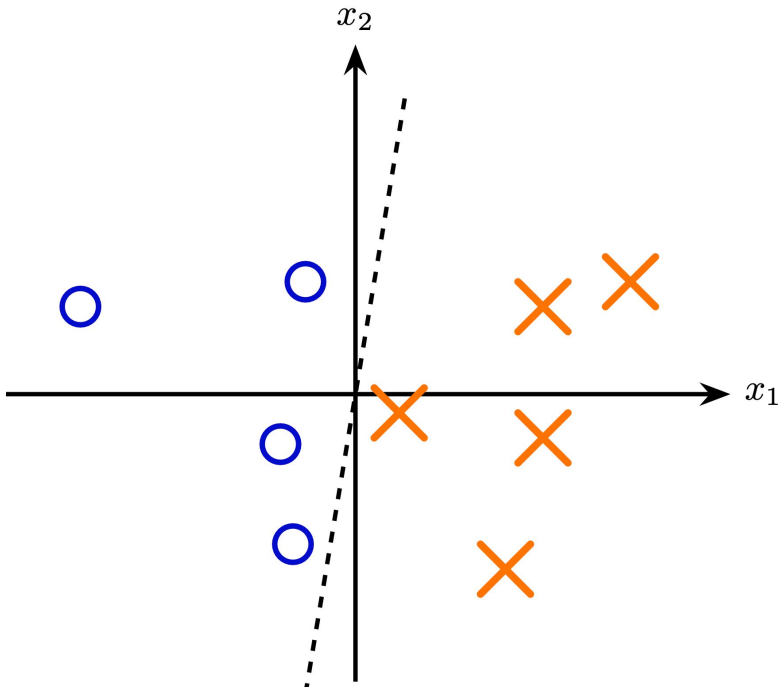
- Error: count of misclassifications

$$\text{empirical_error}(\mathbf{W}) = \sum_{i=1}^{n} 1(f(X_i;\ \mathbf{W}) \neq y_i)$$

Whiteboard: function plots in classification

Learning a perceptron

- Goal: find a **hyperplane** that separates the two groups of points



$\mathbf{W}^\top X$:

- Positive for all positive samples
- Negative for all negative samples

Assume:

- threshold=0
- bias is the weight for an additional input (always 1)

Whiteboard: restate the perceptron

The perceptron algorithm

- Initialize W (random or simply 0)
- For each training example, **update W only if it's misclassified**
  - Positive sample misclassified:

    $$\mathbf{W} \leftarrow \mathbf{W} + X_i$$

  - Negative sample misclassified:

    $$\mathbf{W} \leftarrow \mathbf{W} - X_i$$

Generally:

$$\mathbf{W} \leftarrow \mathbf{W} + y_i X_i$$

$$(y_i \in \{-1, 1\})$$

Whiteboard: demo

Convergence

As long as the classes are **linearly separable**:

The perceptron algorithm is **guaranteed to converge to perfect classification** within a number of misclassifications

Convergence

- Linearly separable:

  $$\forall 1 \leq i \leq n, \;\; y_i(\mathbf{W}^* \cdot X_i) > \gamma \; (\gamma > 0)$$

  Bounded scale:

  $$\|X_i\| \leq R$$

- Maximum number of misclassifications: $\left(\frac{R}{\gamma}\right)^2$

- If not linearly separable, the algorithm does not converge
- Cannot be applied to learn MLPs

Whiteboard: proof (optional)